

Randomized Incremental Construction

Design and Analysis of Algorithms

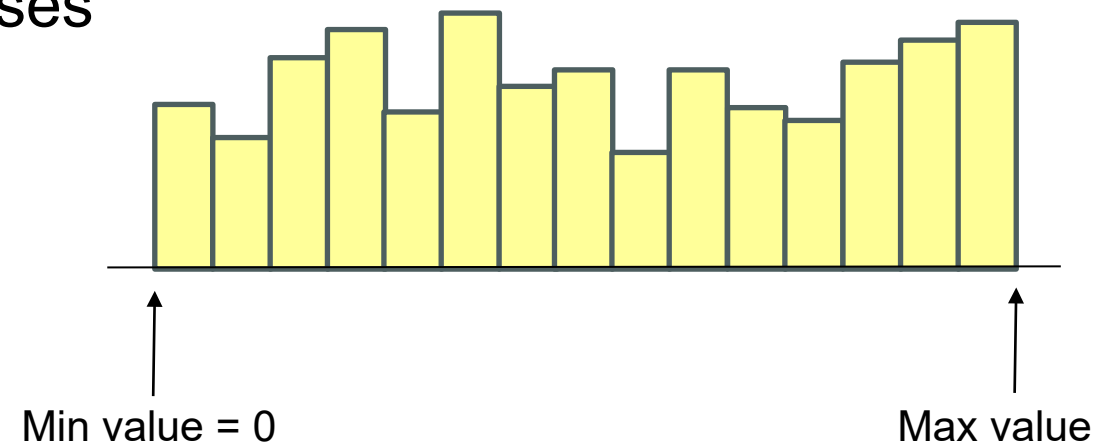
Brian C. Dean

**School of Computing
Clemson University**



Recall: Online Histogram Construction

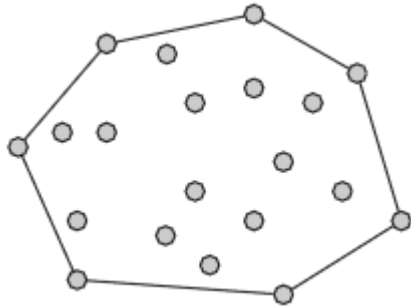
- It's easy to build a histogram out of N values $v_1 \dots v_N$ in $O(N)$ time.
- However, what if the values arrive online and we want to keep the histogram up to date throughout in real time.
- For each new value v_j that arrives...
 - ... normally, just increment appropriate bin count in $O(1)$ time.
 - ... unless v_j is a new maximum, which causes a full re-build in $\Theta(j)$ time!
- Worst case runtime? $\Theta(N^2)$
- Average case if v_i 's arrive in random order? $\Theta(N)$



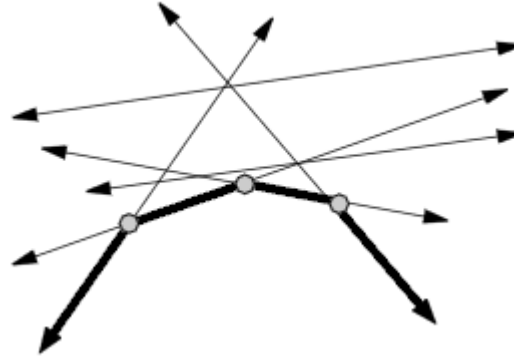
Randomized Incremental Construction

- Maintain solution while adding input elements in random order.
- Sometimes alleviates worst-case behavior, although not always.
 - Insertion sort (an $O(n^2)$ sorting algorithm based on incremental construction) runs in $\Theta(n^2)$ expected time if run on a randomly-permuted array.
- Can often be re-cast through the lens of “average case” performance.
- Will be important as the basis of our preferred method for balancing a binary search tree in a few modules...

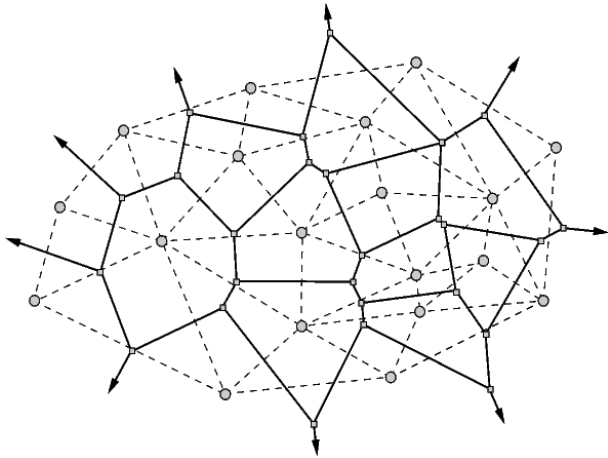
Randomized Incremental Construction in Computational Geometry



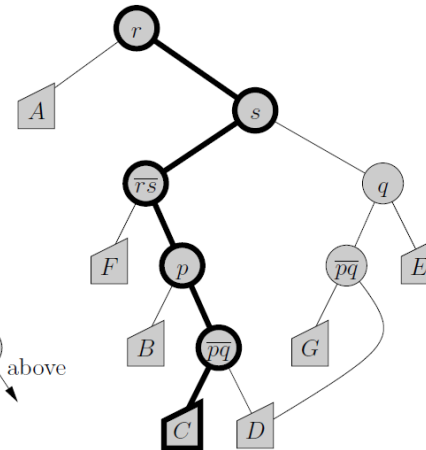
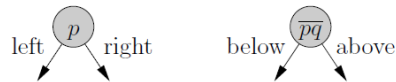
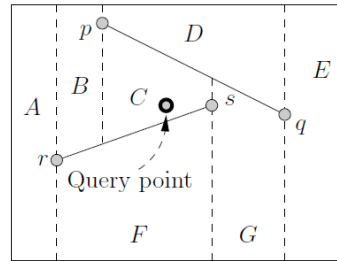
Convex hulls
Upper / lower hulls



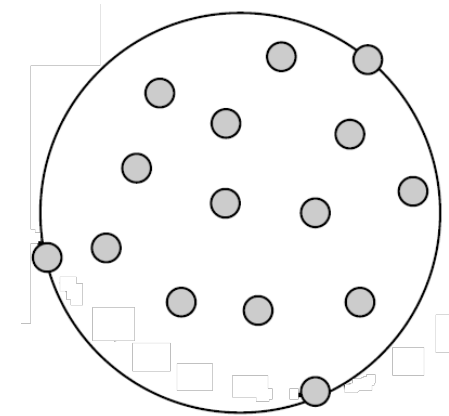
Lower / upper envelopes



Voronoi diagrams
Delaunay triangulations

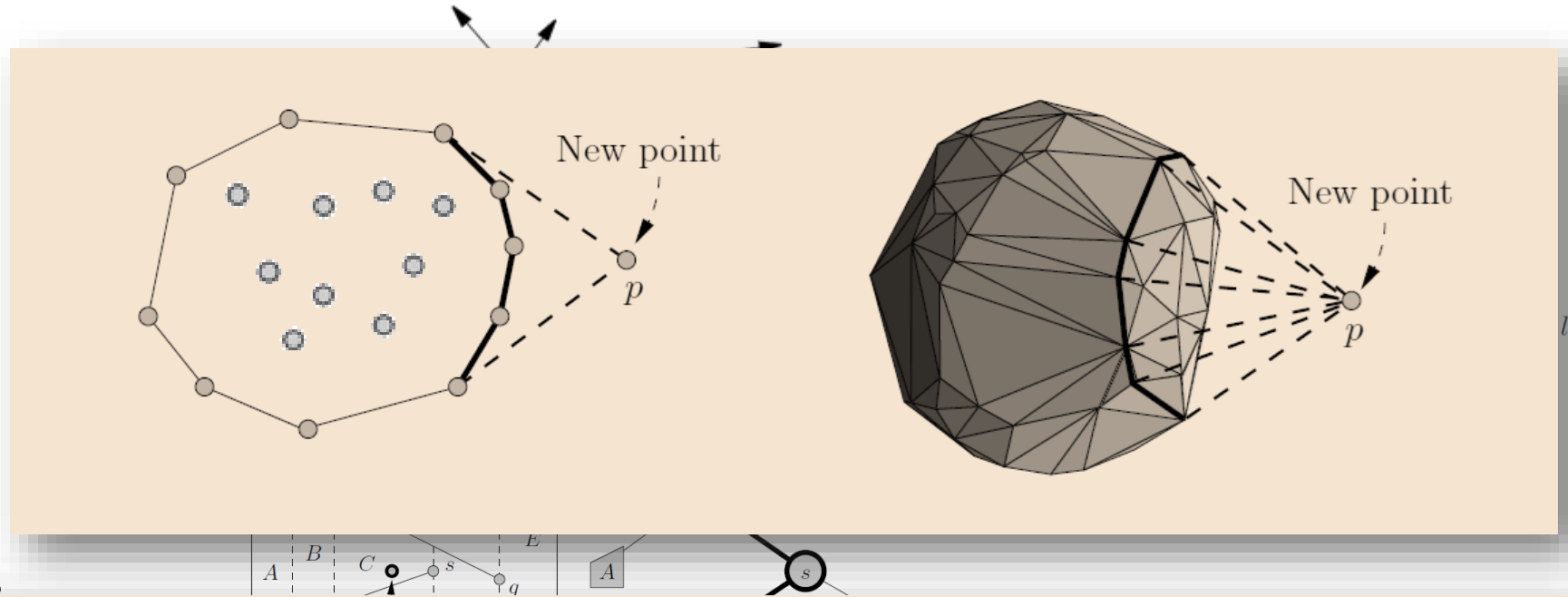
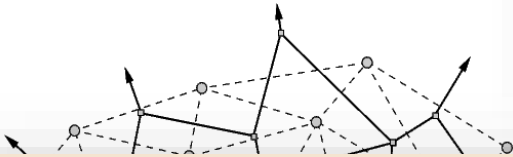
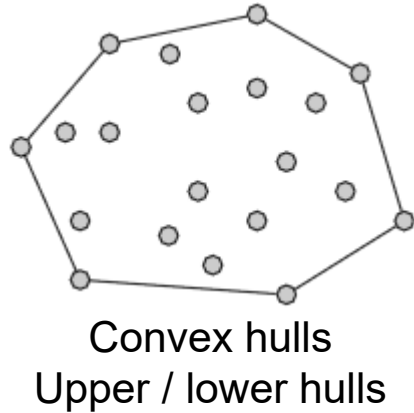


Trapezoidal decompositions
for point location



Smallest enclosing circle

Randomized Incremental Construction in Computational Geometry



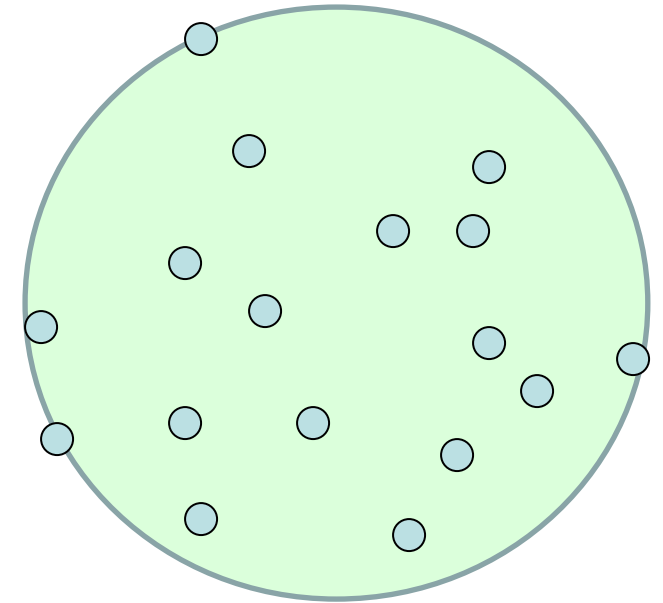
New points added outside current hull cause updates that tend to get more expensive over time, but that also tend to happen less frequently over time.
 $O(n \log n)$ expected running time in 2D and 3D.

Voronoi diagrams
Delaunay triangulations

Trapezoidal decompositions
for point location

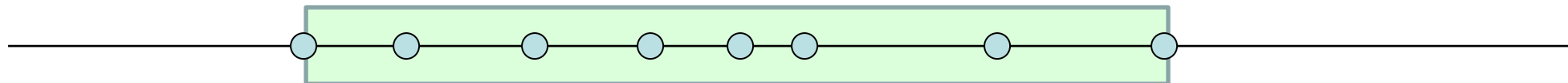
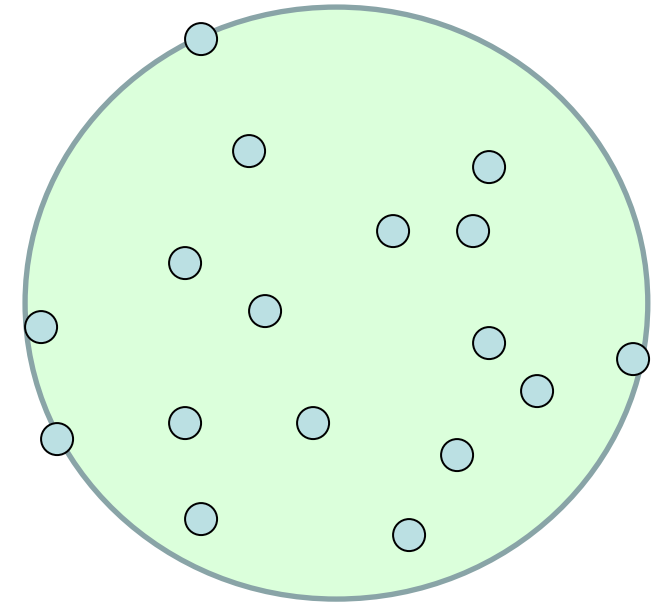
Example: Smallest Enclosing Circle

- Prototypical application: facility location
- A circle is defined by 3 points (or 2 points at opposite ends of a diameter), so there are $\Theta(n^3)$ interesting circles to potentially check.
- “Brute force” running time therefore $O(n^4)$.
- With randomized incremental construction, we’ll be able to achieve $O(n)$ in expectation!



Example: Smallest Enclosing Circle

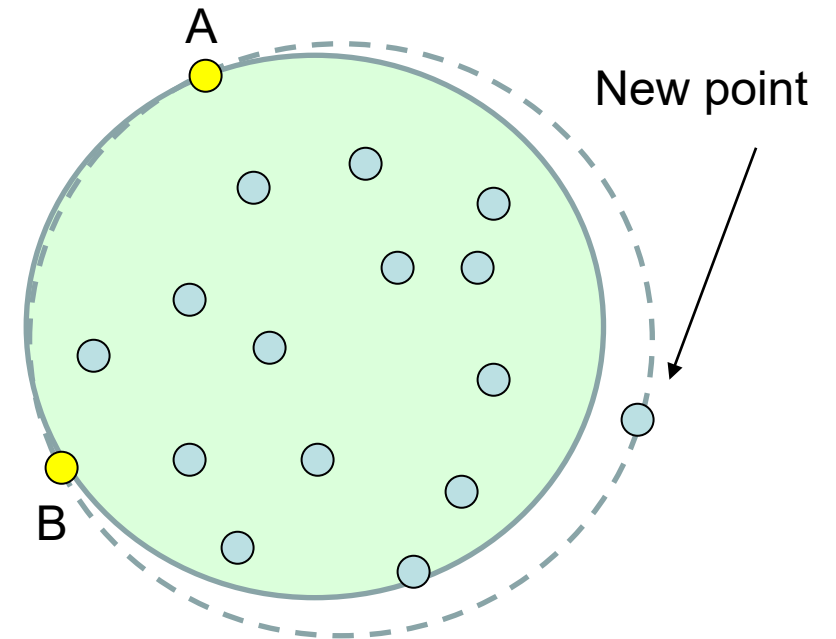
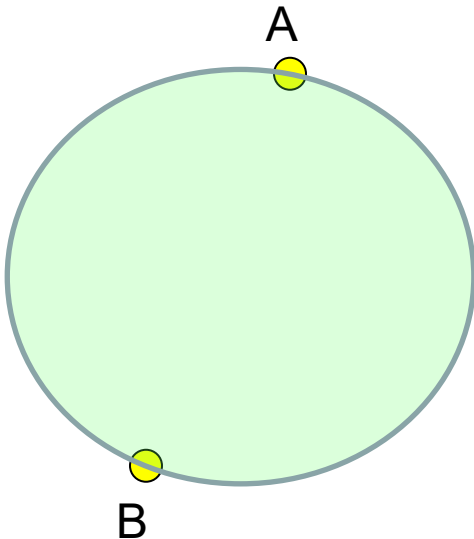
- Prototypical application: facility location
- A circle is defined by 3 points (or 2 points at opposite ends of a diameter), so there are $\Theta(n^3)$ interesting circles to potentially check.
- “Brute force” running time therefore $O(n^4)$.
- With randomized incremental construction, we’ll be able to achieve $O(n)$ in expectation!
 - Can easily generalize to 3D, 4D, etc. (1D boring)



3 Simple Steps...

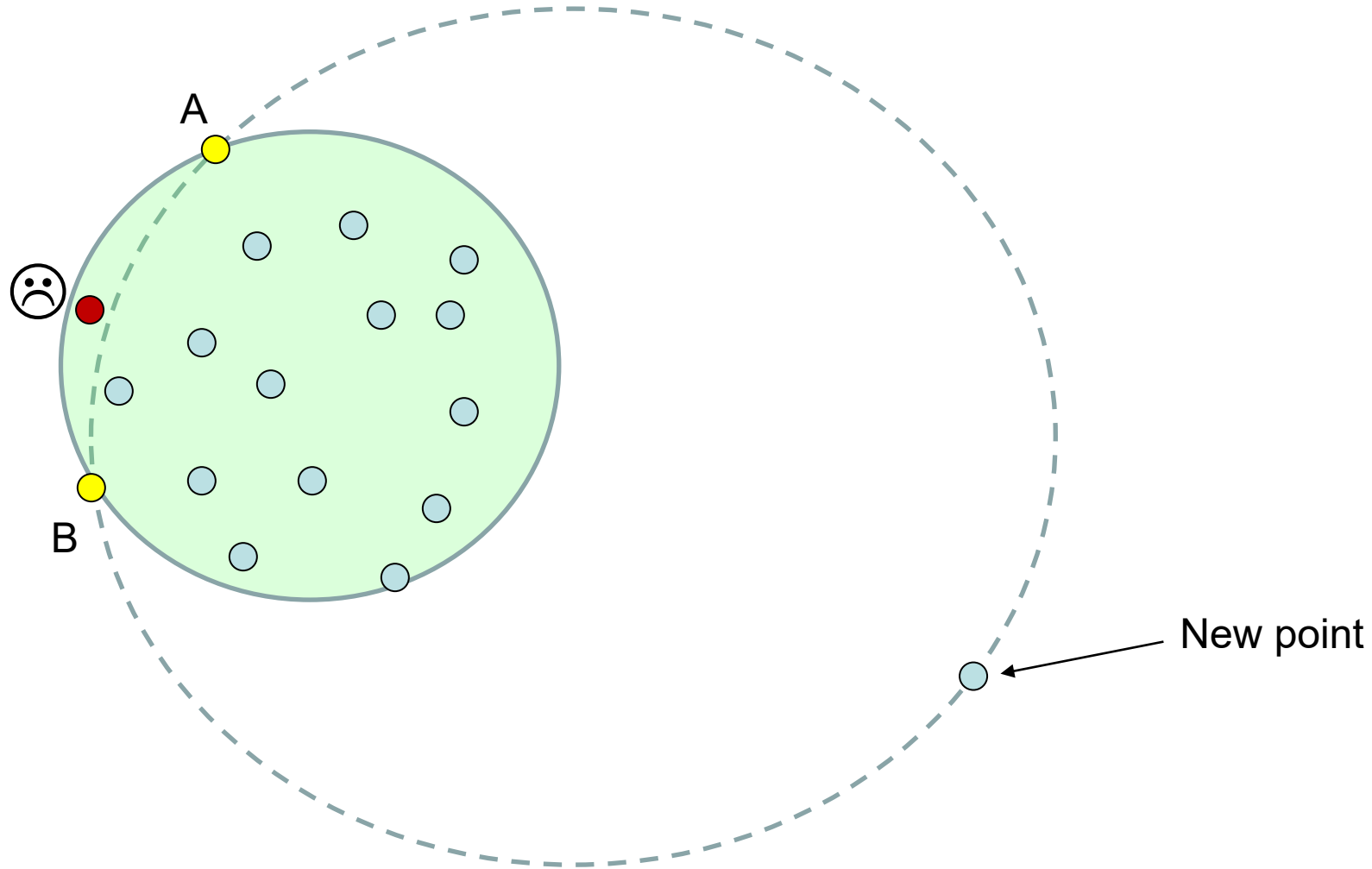
Suppose by “magic” that we already know 2 points A and B on the boundary of an optimal circle. Then to find the optimal circle in $O(n)$ time...

1. Start with a circle having A and B as endpoints of its diameter, and no other points in the picture yet.



2. Process remaining points in sequence, enlarging the circle whenever we process a point outside.

Don't Worry, This Can't Happen... (if Magic is Trustworthy)

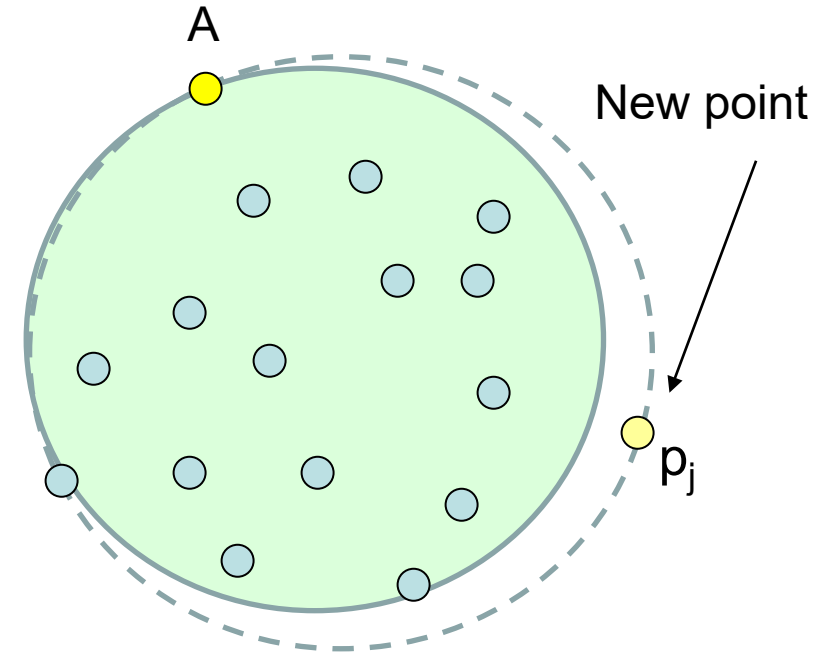


3 Simple Steps...

If we have less “magic” and only know 1 point A on the boundary of an optimal circle. Then:

1. Start with a zero-size circle  around A, and no other points in the picture yet.
2. Process remaining points $p_1 \dots p_{n-1}$ in random order, enlarging whenever we process a point p_j outside

(new circle will need to go through A and p_j , so we use previous $O(j)$ algorithm to determine it)



Easy to see this runs in $O(n)$ expected time with linearity of expectation...

3 Simple Steps...

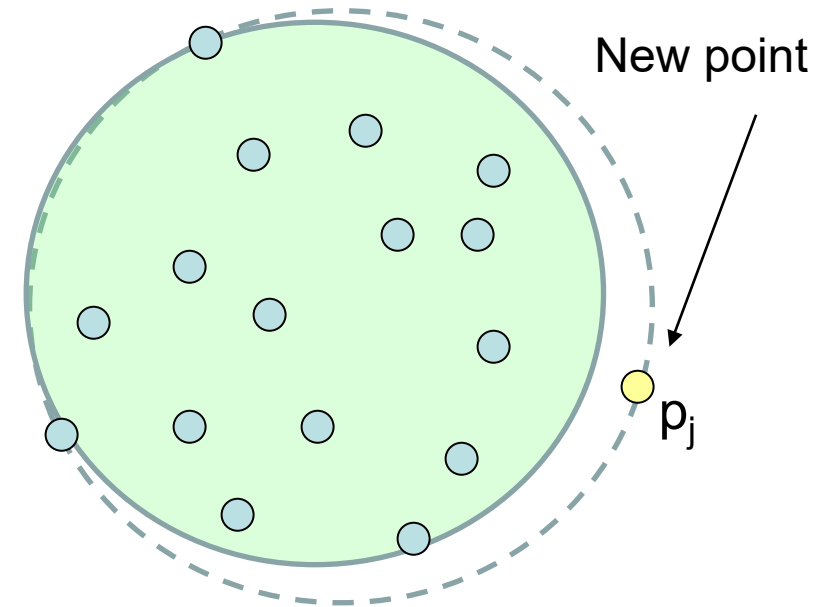
No magic anymore. We don't know any points on the optimal circle to begin with.

1. Start with a lonely zero-size circle , and no points in the picture yet.

2. Process all points $p_1 \dots p_n$ in random order, enlarging whenever we process a point p_j outside

(new circle will need to go through p_j , so we use previous $O(j)$ expected time algorithm to determine it)

This also runs in $O(n)$ expected time, again due to linearity of expectation...



Into 3D and Beyond...

- In 3D, just add another layer. Still $O(n)$ expected time.
- Careful though – factorial dependence on dimension!
- Poor scaling with dimension (the “curse of dimensionality”) is common.
 - E.g., best known algorithms for computing an exact smallest enclosing ball in d dimensions currently have a super-polynomial dependence on d .
 - E.g., 2D and 3D convex hull can be calculated in $O(n \log n)$ time, but in $d > 3$ dimensions the worst-case running time can't improve on $n^{\lfloor d/2 \rfloor}$.
 - This is perhaps concerning news for machine learning...